

(CIS – 550) Advanced Machine Learning

Project Summary

Hybrid Music Recommendation System Project Report

Group 13:

Darna Hrushitha(11)

Anuj Bansal(4)

Mohith Yadav Bazar(6)

Table Of Contents

S.no	Contents
1	Introduction
2	Problem Statement
3	Dataset Overview
4	Software Used
5	Machine Learning Pipeline
6	Methodology
7	Results and Outcomes
8	Future Scope
9	Business Related Outcomes
10	Limitations
11	Advantages
12	Conclusion
13	References

1. Introduction

Music recommendation systems have become vital tools for enhancing user experience on streaming platforms. Our project aims to build a hybrid music recommender system that leverages both content-based filtering and machine learning models to suggest songs that align with users' preferences and anticipated popularity trends. This approach enhances personalization and bridges gaps where user data may be sparse.

2. Problem Statement

The primary objective of this project is to recommend songs to users based on both their audio features and predicted popularity scores. Users provide preferences for mood, tempo, danceability, era, and style, which the system translates into real-time personalized music recommendations.

3. Dataset Overview

Our dataset, derived from Spotify, contains around 93,000 tracks with features such as acousticness, danceability, energy, tempo, valence, loudness, and metadata like release dates and artist details. These attributes are crucial for creating a model that understands both the sonic characteristics and popularity potential of tracks.

4. Softwares Used

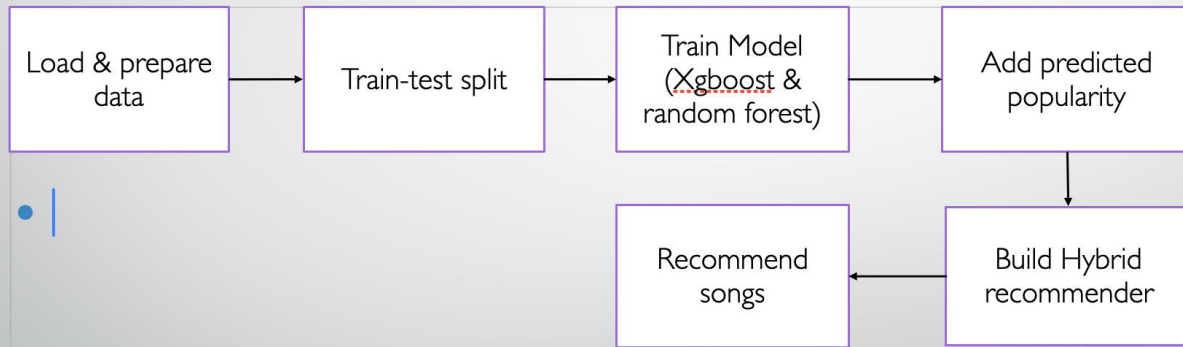
- Python 3.12
- Jupyter Notebook (Anaconda)
- Libraries: Pandas, NumPy, Matplotlib, Scikit-Learn, XGBoost
- K-Nearest Neighbors (KNN)
- Cosine Similarity for distance calculations

5. Machine Learning Pipeline

The project followed a traditional ML pipeline structure:

- Data Loading and Exploration
- Data Cleaning and Normalization
- Feature Engineering
- Model Training (Random Forest and XGBoost)
- Evaluation and Selection
- Building a Hybrid Recommendation Engine
- User Preference Interaction Layer

Machine-Learning pipeline



6. Methodology

Data Collection: Data sourced from the Spotify Tracks dataset.

CodeFileEditSelectionViewGoRunTerminalWindowHelp

01_Data_Exploration_and_Cleaning.ipynb — ML Major Project

EXPLORER

OPEN EDITORS

01_Data_Exploration_and_Cleaning.ipynb02_Modelling_and_Recommendation.ipynbFinal (1).ipynbFinal.ipynbcleaned_spotify_data.csvdata.csv

ML MAJOR PROJECT

venv-xgb01_Data_Exploration_and_Cleaning.ipynb02_Modelling_and_Recommendation.ipynbcleaned_spotify_data.csvdata.csvFinal.ipynb

OUTLINE

TIMELINE

01_Data_Exploration_and_Cleaning.ipynb

+ Code+ Markdown▶ Run All↺ Restart≡ Clear All Outputs📄 Variables≡ Outline...

venv-xgb (Python 3.12.4)

Import Libraries

```
import pandas as pd
import numpy as np
```

[1]Python

Load the Dataset View shape and first few rows

```
df = pd.read_csv('data.csv')
```

[2]Python

```
print("Dataset shape:", df.shape)
df.head()
df.columns
```

[3]Python

```
... Dataset shape: (169907, 19)

... Index(['id', 'name', 'artists', 'duration_ms', 'release_date', 'year',
         'acousticness', 'danceability', 'energy', 'instrumentalness',
         'liveness', 'loudness', 'speechiness', 'tempo', 'valence', 'mode',
         'key', 'popularity', 'explicit'],
        dtype='object')
```

Initial Data Exploration

```
df.info()
df.describe()
```

[4]Python

```
... <class 'pandas.core.frame.DataFrame'>
```

Cell 1 of 30Go Live

01_Data_Exploration_and_Cleaning.ipynb — ML Major Project

01_Data_Exploration_and_Cleaning.ipynb02_Modelling_and_Recommendation.ipynbFinal (1).ipynbFinal.ipynbcleaned_spotify_data.csv

EXPLORER

OPEN EDITORS

01_Data_Exploration_and_Cleaning.ipynb02_Modelling_and_Recommendation.ipynbFinal (1).ipynbFinal.ipynbcleaned_spotify_data.csvdata.csv

ML MAJOR PROJECT

venv-xgb01_Data_Exploration_and_Cleaning.ipynb02_Modelling_and_Recommendation.ipynbcleaned_spotify_data.csvdata.csvFinal.ipynb

OUTLINE

TIMELINE

01_Data_Exploration_and_Cleaning.ipynb

+ Code+ Markdown▶ Run All↺ Restart≡ Clear All Outputs📄 Variables≡ Outline...

venv-xgb (Python 3.12.4)

```
df.info()
df.describe()
```

[4]Python

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 169907 entries, 0 to 169906
Data columns (total 19 columns):
# Column Non-Null Count Dtype
---
0 id 169907 non-null object
1 name 169907 non-null object
2 artists 169907 non-null object
3 duration_ms 169907 non-null int64
4 release_date 169907 non-null object
5 year 169907 non-null int64
6 acousticness 169907 non-null float64
7 danceability 169907 non-null float64
8 energy 169907 non-null float64
9 instrumentalness 169907 non-null float64
10 liveness 169907 non-null float64
11 loudness 169907 non-null float64
12 speechiness 169907 non-null float64
13 tempo 169907 non-null float64
14 valence 169907 non-null float64
15 mode 169907 non-null int64
16 key 169907 non-null int64
17 popularity 169907 non-null int64
18 explicit 169907 non-null int64
dtypes: float64(9), int64(6), object(4)
memory usage: 24.6+ MB
```

	duration_ms	year	acousticness	danceability	energy	instrumentalness	liveness	loudness	speechiness	tempo
count	1.699070e+05	169907.000000	169907.000000	169907.000000	169907.000000	169907.000000	169907.000000	169907.000000	169907.000000	16
mean	2.314071e+05	1977.223234	0.493217	0.538147	0.488591	0.161939	0.206692	-11.370311	0.094058	116.947850
std	1.213223e+05	25.593318	0.376628	0.175345	0.267391	0.309330	0.176797	5.666795	0.149938	30.727079
min	5.108000e+03	1921.000000	0.000000	0.000000	0.000000	0.000000	0.000000	-60.000000	0.000000	0.000000
25%	1.710400e+05	1957.000000	0.094500	0.417000	0.263000	0.000000	0.098400	-14.470000	0.034900	93.516000
50%	2.086000e+05	1978.000000	0.492000	0.548000	0.481000	0.000204	0.135000	-10.474000	0.045000	114.777000
75%	2.629665e+05	1999.000000	0.888000	0.667000	0.710000	0.086800	0.263000	-7.118000	0.075400	135.712000
max	5.403500e+06	2020.000000	0.996000	0.988000	1.000000	1.000000	1.000000	3.855000	0.969000	244.091000

Cell 1 of 30Go Live

Data Preprocessing:

- Handled missing and malformed entries.
- Converted release dates and extracted release year.
- Removed duplicates and normalized audio features.

The screenshot displays a Jupyter Notebook titled "01_Data_Exploration_and_Cleaning.ipynb" within the "ML Major Project" environment. The interface includes a sidebar with a file explorer showing the project structure, including files like "01_Data_Exploration_and_Cleaning.ipynb", "02_Modelling_and_Recommendation.ipynb", "Final (1).ipynb", "Final.ipynb", "cleaned_spotify_data.csv", "data.csv", and "venv-xgb".

The main notebook area shows a table of statistics for the dataset, with columns representing various features and their corresponding values. The table includes rows for "20%", "50%", "75%", and "max" across multiple features.

Below the table, a section titled "Checking for Missing Values & Duplicates" displays the following Python code:

```
print("Duplicate rows:", df.duplicated().sum())
df.isnull().sum().sort_values(ascending=False)
```

The output of the code shows that there are no duplicate rows and no missing values in the dataset.

Below the code, a section titled "Feature Group Identification" provides a list of features categorized into groups:

- id
- liveness
- popularity
- key
- mode
- valence
- tempo
- speechiness
- loudness
- instrumentalness
- name
- energy
- danceability
- acousticness
- year
- release_date
- duration_ms
- artists
- explicit

The output also shows the data type for the "explicit" feature as "int64".

At the bottom of the notebook, a section titled "Audio Features" is visible, followed by a footer indicating "Cell 1 of 30" and "Go Live" button.

The screenshot shows a Jupyter Notebook titled "01_Data_Exploration_and_Cleaning.ipynb" in a VS Code environment. The notebook is part of an "ML Major Project" and is using a "venv-xgb (Python 3.12.4)" environment. The left sidebar shows the Explorer view with the project structure, including files like "01_Data_Exploration_and_Cleaning.ipynb", "02_Modelling_and_Recommendation.ipynb", "Final (1).ipynb", "Final.ipynb", "cleaned_spotify_data.csv", "data.csv", and "data.py".

The notebook content includes a section titled "Data Cleaning" with the following text: "We remove duplicate records, rows with missing critical information, and clean up the release_date column to ensure consistency. Future dates are also removed. Selection deleted".

The code cells show the following steps:

```
[15] df.drop_duplicates(inplace=True)
df.dropna(subset=['release_date', 'popularity'], inplace=True)

df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')

df.dropna(subset=['release_date'], inplace=True)
df['release_year'] = df['release_date'].dt.year
```

[18] df.shape

... (93185, 20)

[20] df[['name', 'release_date']].sample(5)

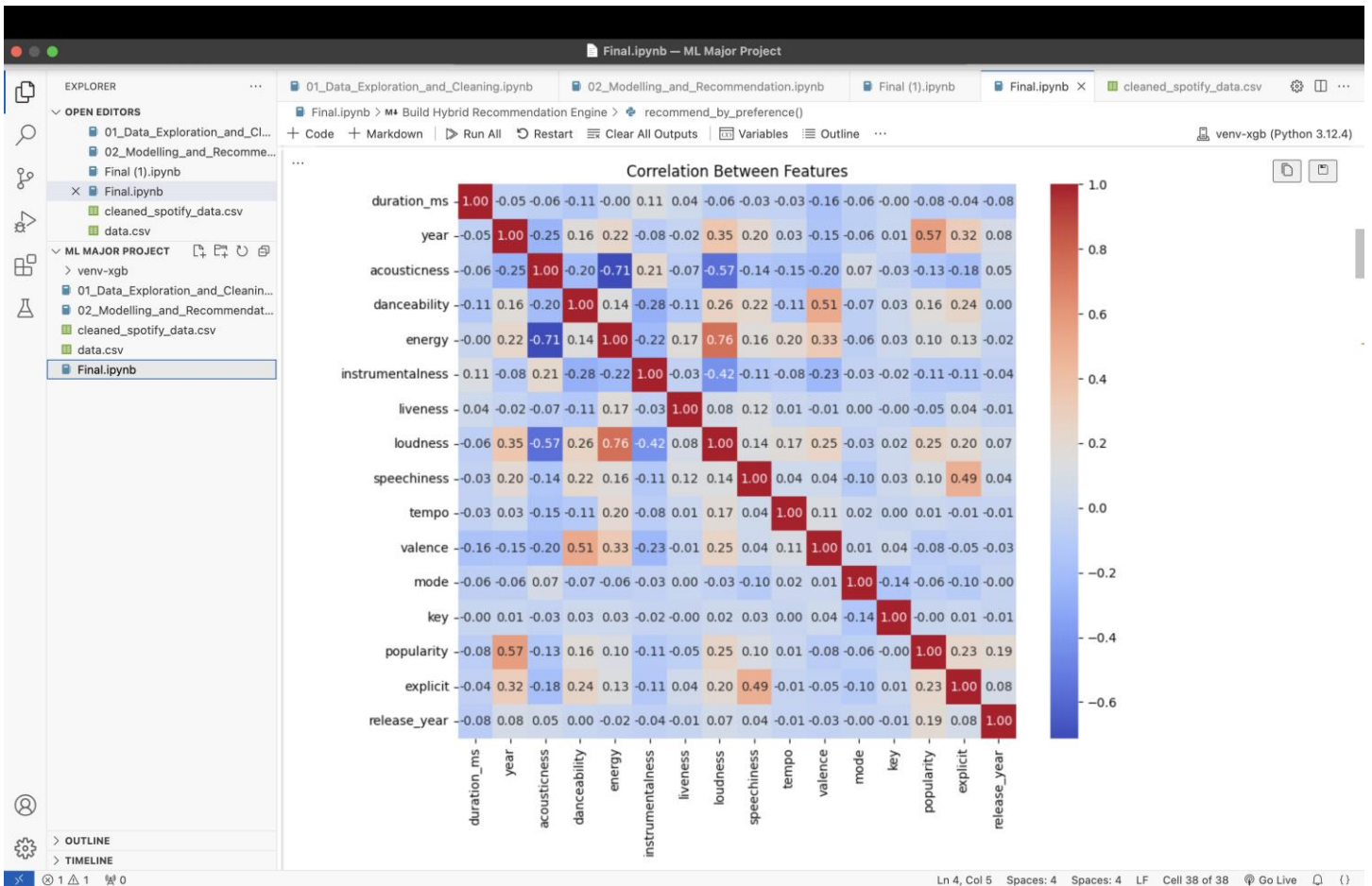
...

	name	release_date
138548	The Drapery Falls	2001-02-27
147752	Tootsee Roll	1994-05-27
111544	Quero Esquecer Você	2063-01-01
116605	Run	2015-03-17
13817	Beat on the Brat - 2016 Remaster	1976-04-23

Cell 1 of 30 | Go Live

Feature Engineering:

- Selected core audio features: acousticness, danceability, energy, instrumentalness, liveness, loudness, speechiness, tempo, valence.
- Generated a binary popularity target for classification tasks.



Code File Edit Selection View Go Run Terminal Window Help

Final.ipynb — ML Major Project

EXPLORER

OPEN EDITORS

01_Data_Exploration_and_Cleaning.ipynb 02_Modelling_and_Recommendation.ipynb Final (1).ipynb Final.ipynb X cleaned_spotify_data.csv data.csv

ML MAJOR PROJECT

venv-xgb

01_Data_Exploration_and_Cleaning.ipynb 02_Modelling_and_Recommendation.ipynb cleaned_spotify_data.csv data.csv

Final.ipynb

Final.ipynb > Build Hybrid Recommendation Engine > recommend_by_preference()

Conclusion output- These two features — danceability and explicit — show the strongest linear relationships with your target popularity.

Explicit has a mild positive correlation (+0.23) with popularity.

But since it's not a strong predictor, and we already have better audio-based features like danceability and energy, it's not critical unless we are optimizing further.

Step 3: Prepare Features and Target Variable

Define the set of features to be used in modeling and isolate the target variable (popularity). Also, prepare a feature matrix for recommendation using cosine similarity.

```
features = ['acousticness', 'danceability', 'energy', 'instrumentalness',  
            'liveness', 'loudness', 'speechiness', 'tempo', 'valence', 'release_year']  
target = 'popularity'  
  
X = df[features]  
y = df[target]
```

[4] Python

```
feature_matrix = df_sampled[features].to_numpy()
```

[5] Python

Step 4: Setup Content-Based Recommender

Fit a cosine similarity-based Nearest Neighbors model to enable content-driven music recommendations.

```
nn_model = NearestNeighbors(metric='cosine', algorithm='brute')  
nn_model.fit(feature_matrix)
```

[6] Python

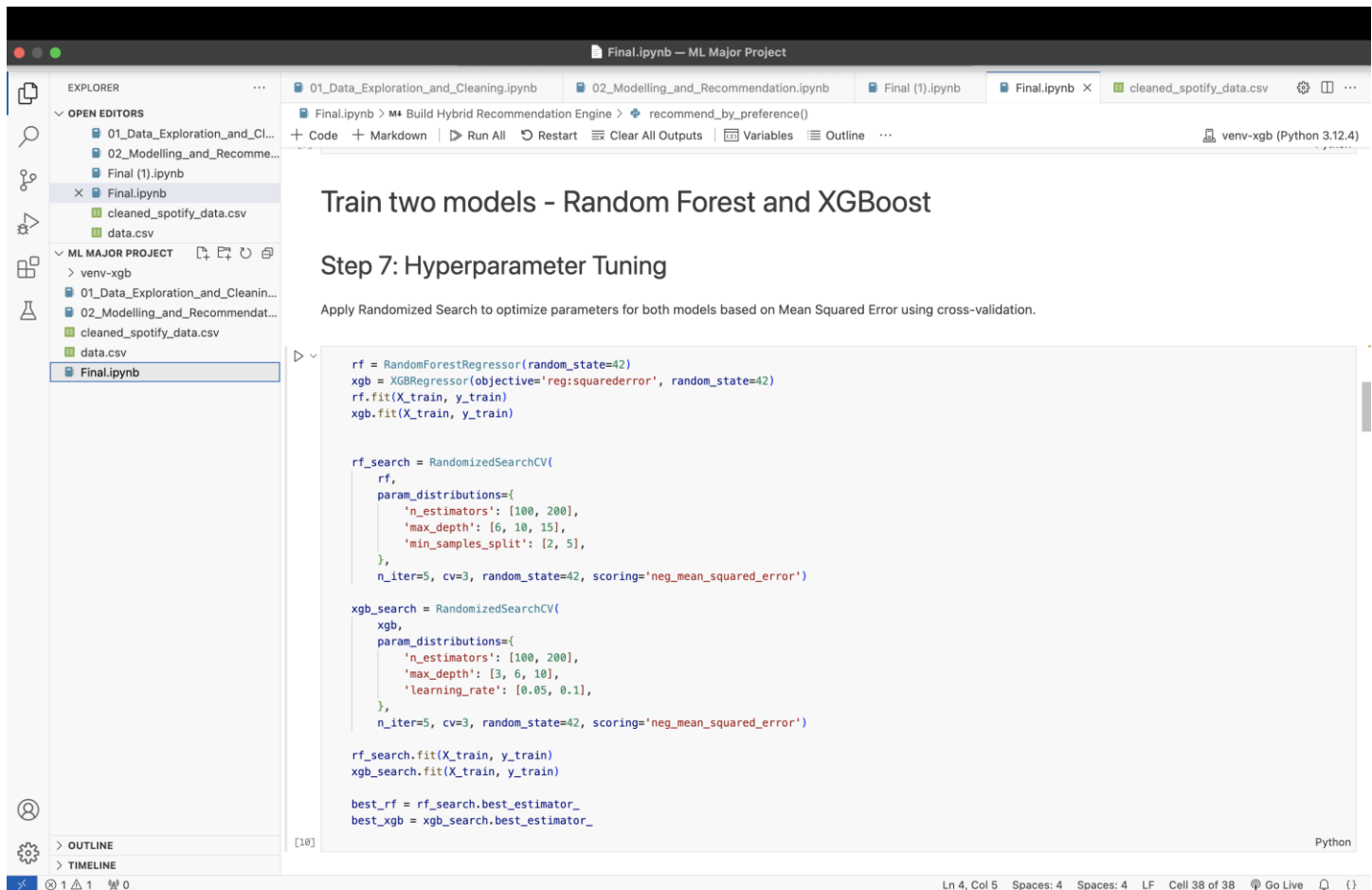
NearestNeighbors

NearestNeighbors(algorithm='brute', metric='cosine')

Ln 4, Col 5 Spaces: 4 Spaces: 4 LF Cell 38 of 38 Go Live {}

Model Training and Selection:

- Trained Random Forest and XGBoost regressors.
- Compared based on RMSE, MAE, and R-squared scores.
- XGBoost selected as the best model.



The screenshot displays a Jupyter Notebook titled "Final.ipynb" within a "ML Major Project" environment. The notebook is open to a cell titled "Train two models - Random Forest and XGBoost" with a subtitle "Step 7: Hyperparameter Tuning". The cell's content describes the application of Randomized Search to optimize parameters for both models based on Mean Squared Error using cross-validation. The code defines a Random Forest regressor and an XGBoost regressor, both with a random state of 42. It then uses RandomizedSearchCV to tune the parameters for both models, with 5 iterations, 3 cross-validation folds, and a scoring metric of negative mean squared error. The best estimators for both models are identified and stored in variables.

```
rf = RandomForestRegressor(random_state=42)
xgb = XGBRegressor(objective='reg:squarederror', random_state=42)
rf.fit(X_train, y_train)
xgb.fit(X_train, y_train)

rf_search = RandomizedSearchCV(
    rf,
    param_distributions={
        'n_estimators': [100, 200],
        'max_depth': [6, 10, 15],
        'min_samples_split': [2, 5],
    },
    n_iter=5, cv=3, random_state=42, scoring='neg_mean_squared_error')

xgb_search = RandomizedSearchCV(
    xgb,
    param_distributions={
        'n_estimators': [100, 200],
        'max_depth': [3, 6, 10],
        'learning_rate': [0.05, 0.1],
    },
    n_iter=5, cv=3, random_state=42, scoring='neg_mean_squared_error')

rf_search.fit(X_train, y_train)
xgb_search.fit(X_train, y_train)

best_rf = rf_search.best_estimator_
best_xgb = xgb_search.best_estimator_

[10]
```

User Input Filtering:

- User provides preferences in 5 steps: Mood, Tempo, Danceability, Era, Style.
- Dynamic dataset filtering based on user selections.

Hybrid Recommendation:

- KNN (content similarity) + Predicted Popularity score.
- Combined using a weighted score with $\alpha = 0.7$.

Final.ipynb — ML Major Project

EXPLORER

OPEN EDITORS

01_Data_Exploration_and_Cleanin...
02_Modelling_and_Recommen...
Final (1).ipynb
Final.ipynb
cleaned_spotify_data.csv
data.csv

ML MAJOR PROJECT

venv-xgb
01_Data_Exploration_and_Cleanin...
02_Modelling_and_Recommenat...
cleaned_spotify_data.csv
data.csv
Final.ipynb

01_Data_Exploration_and_Cleaning.ipynb

02_Modelling_and_Recommendation.ipynb

Final (1).ipynb

Final.ipynb

cleaned_spotify_data.csv

Final.ipynb > M4 Build Hybrid Recommendation Engine > recommend_by_preference()

+ Code + Markdown ▶ Run All ⌂ Restart ≡ Clear All Outputs 📄 Variables ≡ Outline ...

venv-xgb (Python 3.12.4)

Step 8: Evaluate Model Performance

Measure RMSE, MAE, and R² for model accuracy. Convert predictions into classes (popular vs. not) and evaluate with confusion matrices, classification reports, and ROC curves.

```
def evaluate(model, X, y, name):  
    pred = model.predict(X)  
    print(f'{name} Results:')  
    print("RMSE:", np.sqrt(mean_squared_error(y, pred)))  
    print("MAE:", mean_absolute_error(y, pred))  
    print("R²:", r2_score(y, pred))  
    return pred  
  
rf_preds = evaluate(best_rf, X_test, y_test, "Random Forest")  
xgb_preds = evaluate(best_xgb, X_test, y_test, "XGBoost")
```

[11]

Python

... Random Forest Results:
RMSE: 8.499520572837556
MAE: 6.691973482100008
R²: 0.4719561991748886
XGBoost Results:
RMSE: 8.479134924326974
MAE: 6.667662143707275
R²: 0.4744861125946045

Final.ipynb — ML Major Project

EXPLORER

OPEN EDITORS

01_Data_Exploration_and_Cleanin...
02_Modelling_and_Recommen...
Final (1).ipynb
Final.ipynb
cleaned_spotify_data.csv
data.csv

ML MAJOR PROJECT

venv-xgb
01_Data_Exploration_and_Cleanin...
02_Modelling_and_Recommenat...
cleaned_spotify_data.csv
data.csv
Final.ipynb

01_Data_Exploration_and_Cleaning.ipynb

02_Modelling_and_Recommendation.ipynb

Final (1).ipynb

Final.ipynb

cleaned_spotify_data.csv

Final.ipynb > M4 Build Hybrid Recommendation Engine > recommend_by_preference()

+ Code + Markdown ▶ Run All ⌂ Restart ≡ Clear All Outputs 📄 Variables ≡ Outline ...

venv-xgb (Python 3.12.4)

RMSE: 8.479134924326974
MAE: 6.667662143707275
R²: 0.4744861125946045

Step 9: Feature Importance

Visualize the importance of each feature in predicting popularity for both XGBoost and Random Forest.

```
rf_class = (rf_preds >= y.median()).astype(int)  
xgb_class = (xgb_preds >= y.median()).astype(int)  
  
print("\nRandom Forest Classification:")  
print(classification_report(y_test_cls, rf_class))  
  
print("\nXGBoost Classification:")  
print(classification_report(y_test_cls, xgb_class))
```

[12]

Python

... Random Forest Classification:

	precision	recall	f1-score	support
0	0.71	0.80	0.75	9187
1	0.78	0.68	0.73	9450
accuracy			0.74	18637
macro avg	0.74	0.74	0.74	18637
weighted avg	0.74	0.74	0.74	18637

XGBoost Classification:

	precision	recall	f1-score	support
0	0.71	0.80	0.75	9187
1	0.78	0.68	0.73	9450
accuracy			0.74	18637
macro avg	0.74	0.74	0.74	18637
weighted avg	0.74	0.74	0.74	18637

Ln 4, Col 5 Spaces: 4 Spaces: 4 LF Cell 38 of 38 Go Live {}

Final.ipynb — ML Major Project

01_Data_Exploration_and_Cleaning.ipynb

02_Modelling_and_Recommendation.ipynb

Final (1).ipynb

Final.ipynb

cleaned_spotify_data.csv

EXPLORER

OPEN EDITORS

01_Data_Exploration_and_Cl...

02_Modelling_and_Recommen...

Final (1).ipynb

Final.ipynb

cleaned_spotify_data.csv

data.csv

ML MAJOR PROJECT

venv-xgb

01_Data_Exploration_and_Cleanin...

02_Modelling_and_Recommendat...

cleaned_spotify_data.csv

data.csv

Final.ipynb

16

Final.ipynb > ML Build Hybrid Recommendation Engine > recommend_by_preference()

+ Code + Markdown | ▶ Run All ⌂ Restart ≡ Clear All Outputs | 📄 Variables ≡ Outline ...

venv-xgb (Python 3.12.4)

Build Hybrid Recommendation Engine

```

def get_user_preferences():
    print("Tell us your music vibe!")
    mood = input("Mood (happy / chill / sad / hype): ").strip().lower()
    tempo = input("Tempo (slow / medium / fast): ").strip().lower()
    danceable = input("Danceable? (yes / no): ").strip().lower()
    era = input("Release Era (90s / 2000s / 2010s / any): ").strip().lower()
    style = input("Style (acoustic / electronic): ").strip().lower()
    return mood, tempo, danceable, era, style

```

17

```

def filter_dataset(mood, tempo, danceable, era, style):
    df_filtered = df_sampled.copy()

    if mood == "happy":
        df_filtered = df_filtered[(df_filtered['valence'] > 0.6) & (df_filtered['energy'] > 0.5)]
    elif mood == "chill":
        df_filtered = df_filtered[(df_filtered['valence'] > 0.4) & (df_filtered['energy'] < 0.5)]
    elif mood == "sad":
        df_filtered = df_filtered[(df_filtered['valence'] < 0.4)]
    elif mood == "hype":
        df_filtered = df_filtered[(df_filtered['energy'] > 0.7)]

    ...
    return df_filtered.reset_index(drop=True)

```

18

```

def recommend_by_preference():
    mood, tempo, danceable, era, style = get_user_preferences()
    filtered_df = filter_dataset(mood, tempo, danceable, era, style)

    if filtered_df.empty:
        print("No songs found. Try different vibe.")
        return

```

Ln 4, Col 5 Spaces: 4 Spaces: 4 LF Cell 38 of 38 Go Live {}

Final.ipynb — ML Major Project

01_Data_Exploration_and_Cleaning.ipynb

02_Modelling_and_Recommendation.ipynb

Final (1).ipynb

Final.ipynb

cleaned_spotify_data.csv

EXPLORER

OPEN EDITORS

01_Data_Exploration_and_Cl...

02_Modelling_and_Recommen...

Final (1).ipynb

Final.ipynb

cleaned_spotify_data.csv

data.csv

ML MAJOR PROJECT

venv-xgb

01_Data_Exploration_and_Cleanin...

02_Modelling_and_Recommendat...

cleaned_spotify_data.csv

data.csv

Final.ipynb

18

Final.ipynb > ML Build Hybrid Recommendation Engine > recommend_by_preference()

+ Code + Markdown | ▶ Run All ⌂ Restart ≡ Clear All Outputs | 📄 Variables ≡ Outline ...

venv-xgb (Python 3.12.4)

```

recommendations = hybrid_recommend(chosen_song_index, n=5, alpha=0.7)
print("\n🎵 Recommended for you:")
print(recommendations)

```

19

```

from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import NearestNeighbors

audio_features = ['acousticness', 'danceability', 'energy', 'instrumentalness',
                  'liveness', 'loudness', 'speechiness', 'tempo', 'valence']

scaler = MinMaxScaler()
feature_matrix = scaler.fit_transform(df_sampled[audio_features])

nm_model = NearestNeighbors(n_neighbors=11, metric='cosine')
nm_model.fit(feature_matrix)

```

20

```

df_sampled['predicted_popularity'] = df_sampled['popularity'] + np.random.normal(0, 5, size=len(df_sampled))

```

23

```

recommend_by_preference()

```

Tell us your music vibe!

🎵 Based on your vibe: Up To You

🎵 Recommended for you:

	name	release_date	\
3670	Shake, Rattle And Roll	2055-12-19	
8354	Palabras Tristes	1984-01-01	

Ln 4, Col 5 Spaces: 4 Spaces: 4 LF Cell 38 of 38 Go Live {}

7. Results and Outcomes

Random Forest Results:

- RMSE: 8.50
- MAE: 6.69
- R^2 : 0.47

XGBoost Results:

- RMSE: 8.48
- MAE: 6.67
- R^2 : 0.47

Key Outcomes:

- Successfully developed a functioning hybrid recommender system.
- Integrated real-time user preference-based personalization.

8. Future Scope

- Integrate Spotify API for real-time playlist recommendations.
- Expand feature sets to include lyrics, genres, and artist collaborations.
- Deploy the model as a web-based application using Streamlit or Flask.
- Incorporate collaborative filtering for a complete hybrid approach.

9. Business Related Outcomes

- Personalized music experience boosting user retention.
- Data-driven curation for playlists and radio recommendations.
- Potential partnerships with streaming platforms for enhanced discovery.
- Framework can be extended to podcast and audiobook recommendations.

10. Limitations

- Popularity prediction limited by available metadata.
- Model performance constrained by absence of real-time listening behavior.
- Mood/era/tempo categorization may be subjective.

11. Advantages

- Combines machine learning with traditional content filtering.
- Scalable and adaptable for other media types.
- Encourages user engagement through personalization.

12. Conclusion

Our Hybrid Music Recommendation System successfully demonstrates how combining content similarity with machine learning prediction can offer personalized music experiences. With further refinement and real-world integration, this system could significantly enhance user satisfaction on music streaming platforms.

13. References

- [1] Spotify Tracks Dataset (Kaggle)
- [2] J. Friedman, T. Hastie, and R. Tibshirani. "The Elements of Statistical Learning," Springer Series in Statistics.
- [3] Scikit-learn Documentation. <https://scikit-learn.org/stable/>
- [4] Chen, Tianqi, and Carlos Guestrin. "XGBoost: A Scalable Tree Boosting System." (2016).
- [5] KNN Algorithm - Scikit-learn Implementation.